# Theoretical Questions

## Question 1: Decision Tree Construction with Manual Calculation


### Training Dataset (6 samples)

| ID | Feature A | Feature B | Feature C | Label |
|---|---|---|---|---|
| 1 | Yes | High | Low | 1 |
| 2 | No | High | Medium | 0 |
| 3 | Yes | Low | High | 1 |
| 4 | No | Medium | Low | 0 |
| 5 | Yes | High | Medium | 0 |
| 6 | No | Medium | High | 0 |

Total: $6$ samples, with

- $\#\{1\} = 2$
- $\#\{0\} = 4$

So

$$p(1) = \frac{2}{6} = \frac{1}{3}, \qquad p(0) = \frac{4}{6} = \frac{2}{3}.$$

### Validation Dataset (2 samples)

| ID | Feature A | Feature B | Feature C | Label |
|---|---|---|---|---|
| V1 | Yes | High | Medium | 1 |
| V2 | No | Low | Low | 0 |

### Entropy

$$H(S) = -\sum_{i=1}^{k} p_i \log_2(p_i).$$

### Gini

$$G(S) = 1 - \sum_{i=1}^{k} p_i^2.$$

**Weighted split entropy** For a split into subsets $S_1, S_2, \ldots, S_m$:

$$H_{\text{split}} = \sum_{j=1}^{m} \frac{|S_j|}{|S|} H(S_j).$$

### Information Gain

$$IG = H(S) - H_{\text{split}}.$$

(For Gini, we minimize the weighted $G_{\text{split}}$ analogously.)

## 1) Root Node Split

### Root Impurity

Subset $A = \text{Yes}$ (IDs 1,3,5) Labels: $(1, 1, 0)$, so

$$p(1) = \frac{2}{3}, \quad p(0) = \frac{1}{3}.$$

Entropy:

$$H(S_{A=\text{Yes}}) = -\frac{2}{3}\log_2\left(\frac{2}{3}\right) - \frac{1}{3}\log_2\left(\frac{1}{3}\right) \approx 0.9183.$$

Gini:

$$G(S_{A=\text{Yes}}) = 1 - \left(\frac{2}{3}\right)^2 - \left(\frac{1}{3}\right)^2 = \frac{4}{9} \approx 0.4444.$$

Subset $A = \text{No}$ (IDs 2,4,6) Labels: $(0, 0, 0)$ (pure), so

$$H(S_{A=\text{No}}) = 0, \qquad G(S_{A=\text{No}}) = 0.$$

Weighted impurity after split on $A$ Entropy:

$$H_{\text{split}}(A) = \frac{3}{6}(0.9183) + \frac{3}{6}(0) = 0.45915.$$

Information Gain:

$$IG(A) = 0.9183 - 0.45915 = 0.45915.$$

Gini:

$$G_{\text{split}}(A) = \frac{3}{6}(0.4444) + \frac{3}{6}(0) = 0.2222.$$

**Split on Feature B (High/Medium/Low)**

$B = \text{High}$ (IDs 1,2,5)
Labels $(1, 0, 0)$, so

$$p(1) = \frac{1}{3}, \quad p(0) = \frac{2}{3}.$$

Thus

$$H(S_{B=\text{High}}) \approx 0.9183, \qquad G(S_{B=\text{High}}) \approx 0.4444.$$

$B = \text{Medium}$ (IDs 4,6)
Labels $(0, 0)$ (pure):

$$H = 0, \quad G = 0.$$

$B = \text{Low}$ (ID 3)
Labels $(1)$ (pure):

$$H = 0, \quad G = 0.$$

Weighted impurity after split on $B$
Entropy:

$$H_{\text{split}}(B) = \frac{3}{6}(0.9183) + \frac{2}{6}(0) + \frac{1}{6}(0) = 0.45915.$$

Information Gain:

$$IG(B) = 0.9183 - 0.45915 = 0.45915.$$

Gini:

$$G_{\text{split}}(B) = \frac{3}{6}(0.4444) = 0.2222.$$

**Split on Feature C (Low/Medium/High)**
$C = \text{Low}$ (IDs 1,4) Labels $(1, 0)$:

$$H(S_{C=\text{Low}}) = 1, \qquad G(S_{C=\text{Low}}) = 0.5.$$

$C = \text{Medium}$ (IDs 2,5)
Labels $(0, 0)$ (pure):

$$H = 0, \quad G = 0.$$

$C = \text{High}$ (IDs 3,6)
Labels $(1, 0)$:

$$H(S_{C=\text{High}}) = 1, \qquad G(S_{C=\text{High}}) = 0.5.$$

Weighted impurity after split on $C$
Entropy:

$$H_{\text{split}}(C) = \frac{2}{6}(1) + \frac{2}{6}(0) + \frac{2}{6}(1) = \frac{4}{6} = 0.6667.$$

Information Gain:

$$IG(C) = 0.9183 - 0.6667 = 0.2516.$$

Gini:

$$G_{\text{split}}(C) = \frac{2}{6}(0.5) + \frac{2}{6}(0) + \frac{2}{6}(0.5) = \frac{2}{6} = 0.3333.$$

**Best Root Split Decision**

- By **Information Gain**: $IG(A) = IG(B) = 0.45915$ (tie), both better than $IG(C) = 0.2516$.
- By **Gini** (minimize): $G_{\text{split}}(A) = G_{\text{split}}(B) = 0.2222$ (tie), both better than $G_{\text{split}}(C) = 0.3333$.

Best root split is a tie between **Feature A** and **Feature B**. Below, I decided to choose **Feature A** as the root (binary split).

## 2) Fully Grown Decision Tree Construction

**Root: Split on Feature A**

Branch 1: $A = \text{No}$
IDs 2,4,6 have labels $(0, 0, 0)$, so this node is pure:

$$\boxed{\text{If } A = \text{No} \Rightarrow \hat{y} = 0.}$$

Branch 2: $A = \text{Yes}$
IDs 1,3,5 have labels $(1, 1, 0)$, not pure.

Node impurity:

$$H \approx 0.9183, \qquad G \approx 0.4444.$$

Now consider splitting this node using remaining features $B$ or $C$.

Split $A = \mathrm{Yes}$ node by Feature C

Within $A = \mathrm{Yes}$:

- $C = \mathrm{Low}$: ID 1 has label $1$ (pure)
- $C = \mathrm{High}$: ID 3 has label $1$ (pure)
- $C = \mathrm{Medium}$: ID 5 has label $0$ (pure)

Thus the weighted impurity after the split is

$$H_{\text{split}} = 0, \qquad G_{\text{split}} = 0,$$

which is a perfect split.

So the fully grown tree is:

- If $A = \mathrm{No}$: predict $0$
- If $A = \mathrm{Yes}$:
  - If $C = \mathrm{Low}$: predict $1$
  - If $C = \mathrm{High}$: predict $1$
  - If $C = \mathrm{Medium}$: predict $0$

## 3) Pruning with Reduced Error Pruning (REP)

Validation points:

1. V1: $(A = \mathrm{Yes}, C = \mathrm{Medium})$, true label $1$
   Tree prediction: for $A = \mathrm{Yes}$ and $C = \mathrm{Medium}$, predict $0 \Rightarrow$ incorrect.
2. V2: $(A = \mathrm{No})$, true label $0$
   Tree prediction: for $A = \mathrm{No}$, predict $0 \Rightarrow$ correct.

Validation error (misclassification rate):

$$\mathrm{Err}_{\text{full}} = \frac{1}{2} = 0.5.$$

**Simulate pruning at node "$A = \mathrm{Yes}$"**

Collapse the entire subtree under $A = \mathrm{Yes}$ into a single leaf node using the **majority vote** among training samples with $A = \mathrm{Yes}$.

Training labels where $A = \mathrm{Yes}$ are $(1, 1, 0)$, so majority class is $1$.

Pruned tree:

- If $A = \mathrm{No}$: predict $0$
- If $A = \mathrm{Yes}$: predict $1$

**Validation error after pruning**

- V1: $A = \mathrm{Yes} \Rightarrow$ predict $1$ (correct)
- V2: $A = \mathrm{No} \Rightarrow$ predict $0$ (correct)

So

$$\mathrm{Err}_{\text{pruned}} = \frac{0}{2} = 0.$$

**REP decision:** prune, since validation error decreases from $0.5$ to $0$.

## 4) Comparison

**Why did the fully grown tree fail on Validation set?**

Compare:

- Training sample 5: $(A = \mathrm{Yes}, B = \mathrm{High}, C = \mathrm{Medium}) \Rightarrow 0$
- Validation sample V1: $(A = \mathrm{Yes}, B = \mathrm{High}, C = \mathrm{Medium}) \Rightarrow 1$

The fully grown tree created a very specific leaf rule:

$$A = \mathrm{Yes},\ C = \mathrm{Medium} \Rightarrow 0$$

based on only **one training observation** (ID 5), so it effectively **memorized noise**. When the validation set contains the same feature combination with the opposite label, the tree misclassifies it (overfitting).

**How does pruning impact complexity and the bias-variance tradeoff?**

- Fully grown tree: more splits, more complex
  $\Rightarrow$ lower bias but higher variance (sensitive to small-sample quirks)

- Pruned tree: fewer splits, simpler model
  $\Rightarrow$ higher bias but lower variance (better generalization)

In this problem, pruning reduces variance enough that validation performance improves.

# Question 2: Logistic Regression: Optimization and Regularization

## 1. Loss Function Derivation (Maximum Likelihood Estimation)

### 1.1 Model Definition

Logistic regression models the probability of class $1$ as

$$\hat{y}_i = p(y_i = 1 \mid x_i) = \sigma(\theta^T x_i), \qquad \sigma(z) = \frac{1}{1 + e^{-z}}.$$

### 1.2 Likelihood Function

For a single observation $(x_i, y_i)$:

$$p(y_i \mid x_i; \theta) = \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}.$$

Assuming independence of samples, the likelihood is

$$L(\theta) = \prod_{i=1}^{n} \hat{y}_i^{y_i} (1 - \hat{y}_i)^{1 - y_i}.$$

### 1.3 Negative Log-Likelihood (Binary Cross-Entropy)

Taking logs and negating:

$$J(\theta) = -\sum_{i=1}^{n} \left[ y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i) \right].$$

This is the **Binary Cross-Entropy loss**.

## 2. Gradient Descent: Manual Calculation

### 2.1 Dataset and Initialization

| Sample | $x = [x_1, x_2]^T$ | $y$ |
|---|---|---|
| 1 | $[1, 0]^T$ | $1$ |
| 2 | $[0, 1]^T$ | $0$ |

Initial weights and learning rate:

$$\theta^{(0)} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}, \qquad \eta = 1.0$$

### 2.2 Predictions

For both samples:

$$z_i = \theta^T x_i = 0 \quad \Rightarrow \quad \hat{y}_i = \sigma(0) = 0.5$$

### 2.3 Gradient Computation

Gradient formula:

$$\nabla_\theta J = \sum_{i=1}^{n} (\hat{y}_i - y_i) x_i$$

- Sample 1:

$$(0.5 - 1) \begin{bmatrix} 1 \\ 0 \end{bmatrix} = \begin{bmatrix} -0.5 \\ 0 \end{bmatrix}$$

- Sample 2:

$$(0.5 - 0) \begin{bmatrix} 0 \\ 1 \end{bmatrix} = \begin{bmatrix} 0 \\ 0.5 \end{bmatrix}$$

Total gradient:

$$\nabla_\theta J = \begin{bmatrix} -0.5 \\ 0.5 \end{bmatrix}$$

### 2.4 Gradient Descent Update

$$\theta^{(1)} = \theta^{(0)} - \eta \nabla_\theta J = \begin{bmatrix} 0.5 \\ -0.5 \end{bmatrix}$$

## 3. Regularization (L2 vs L1)

### 3.1 Regularized Objective

$$J_{\text{reg}}(\theta) = J(\theta) + \lambda R(\theta)$$

### 3.2 Ridge Regression (L2)

Regularizer:

$$R(\theta) = \|\theta\|_2^2 = \sum_j \theta_j^2$$

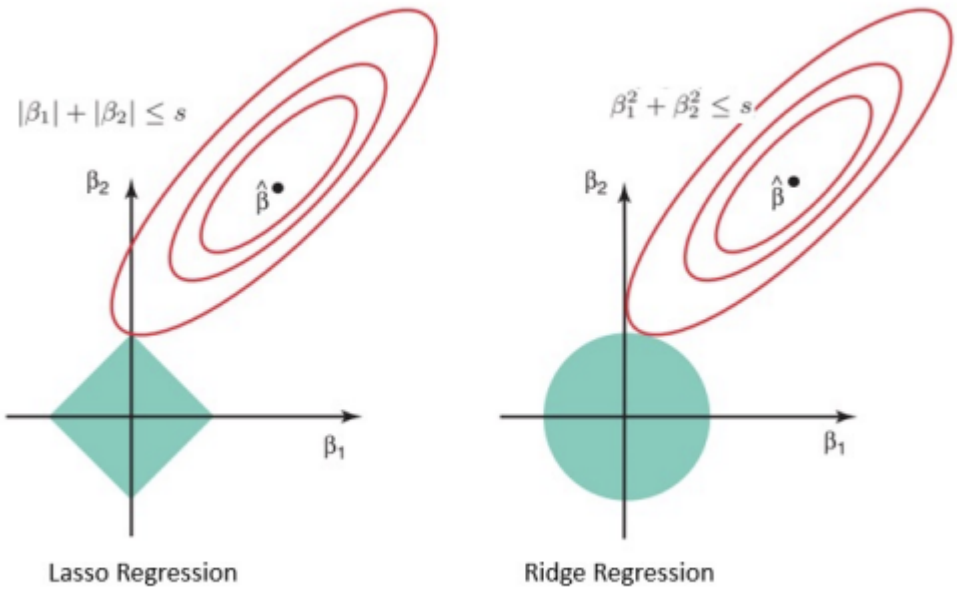
Gradient:

$$\frac{\partial J_{\text{reg}}}{\partial \theta_j} = \frac{\partial J}{\partial \theta_j} + 2\lambda\theta_j$$

Update rule:

$$\theta_j \leftarrow (1 - 2\eta\lambda)\theta_j - \eta\frac{\partial J}{\partial \theta_j}$$

**Interpretation (Weight Decay):**
Each update multiplicatively shrinks $\theta_j$ toward zero.




### 3.3 Lasso Regression (L1)

Regularizer:

$$R(\theta) = \|\theta\|_1 = \sum_j |\theta_j|$$

**Geometric Explanation:**

- L2 constraint ⟶ **circle**
- L1 constraint ⟶ **diamond**

The diamond has sharp corners on the axes, so optimal solutions often lie on axes, forcing some coefficients to be exactly zero.

**Conclusion:**

- L1 ⟶ sparse solutions (feature selection)
- L2 ⟶ shrinkage without sparsity

## 4. Decision Boundary

### 4.1 Given Parameters

$$\theta = \begin{bmatrix} 2 \\ -1 \end{bmatrix}, \qquad b = -1$$

### 4.2 Decision Boundary Equation

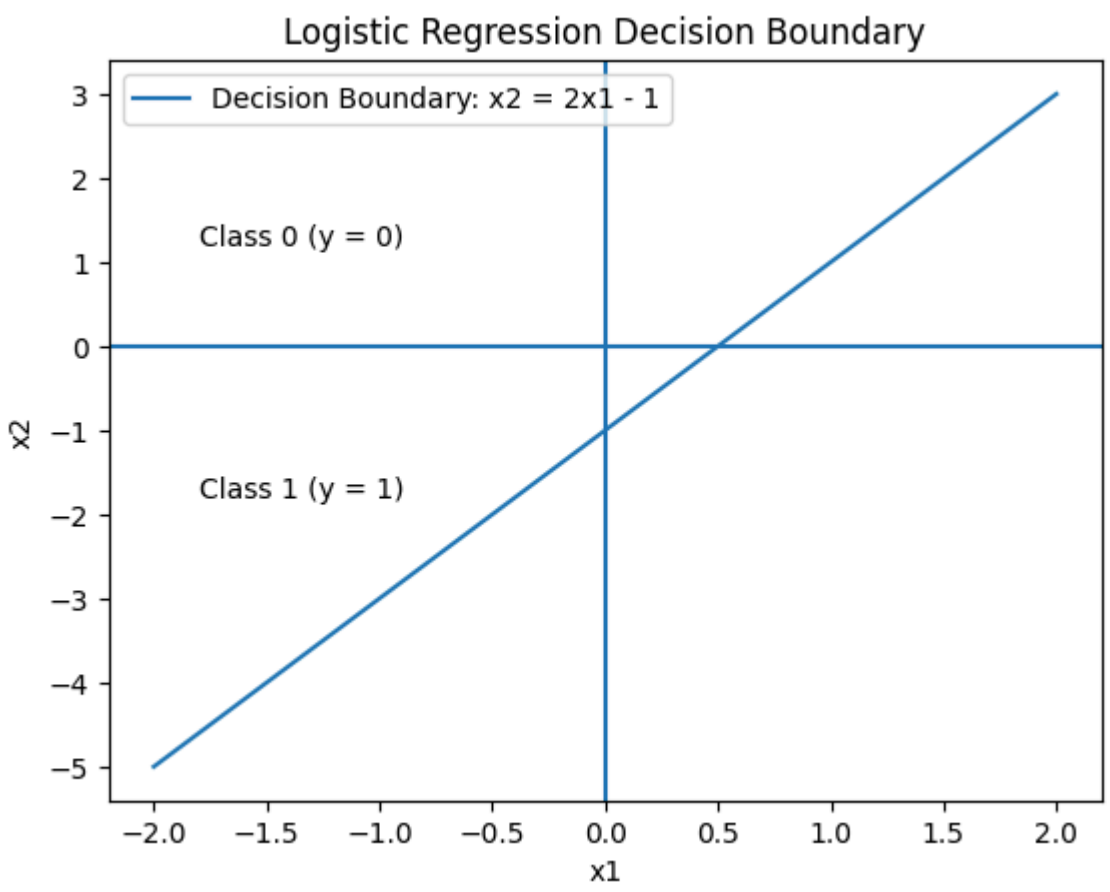
The boundary satisfies:

$$\theta^T x + b = 0$$

Thus:

$$2x_1 - x_2 - 1 = 0 \quad \Rightarrow \quad \boxed{x_2 = 2x_1 - 1}$$

```
import numpy as np
import matplotlib.pyplot as plt

# Parameters (4.1)
theta = np.array([2, -1])
b = -1

# Decision boundary (4.2)
x1 = np.linspace(-2, 2, 400)
x2 = 2 * x1 - 1

# Plot Decision boundary
plt.figure()
plt.plot(x1, x2, label="Decision Boundary: x2 = 2x1 - 1")

# Axes
plt.axhline(0)
plt.axvline(0)

# Labels and title
plt.xlabel("x1")
plt.ylabel("x2")
plt.title("Logistic Regression Decision Boundary")

# Class regions
plt.text(-1.8, -1.8, "Class 1 (y = 1)")
plt.text(-1.8, 1.2, "Class 0 (y = 0)")

plt.legend()
plt.show()
```




**4.3 Classification Regions**

Predict **Class 1** if:

$$2x_1 - x_2 - 1 \geq 0 \quad \Leftrightarrow \quad x_2 \leq 2x_1 - 1$$

Predict **Class 0** otherwise.

**Summary:**

- Straight-line boundary with slope $2$
- Class 1 lies **below** the line
- Class 0 lies **above** the line

## Question 3: Mixture of Linear Regressions (EM Algorithm)

We model stock returns using a **mixture of two linear regressions without intercept**:

$$y_i = \beta_k x_i + \varepsilon_i, \qquad \varepsilon_i \sim \mathcal{N}(0, \sigma^2),$$

where:

- $k \in \{1, 2\}$ is a **latent market regime**

- $\pi_1 = \pi_2 = 0.5$
- $\sigma^2 = 1$ (fixed)

Given Data

| Sample $i$ | $x_i$ | $y_i$ |
|---|---|---|
| 1 | 1 | 1 |
| 2 | 1 | -1 |
| 3 | 1 | 0 |

Initial Parameters

- Regime 1: $\beta_1^{(0)} = 1$
- Regime 2: $\beta_2^{(0)} = -1$
- Variance: $\sigma^2 = 1$
- Mixing coefficients: $\pi_1 = \pi_2 = 0.5$

## 1. E-Step: Posterior Probabilities

We compute the responsibility:

$$\gamma_{ik} = \frac{\pi_k \, p(y_i \mid x_i, \beta_k)}{\sum_{j=1}^{2} \pi_j \, p(y_i \mid x_i, \beta_j)}$$

Since $\sigma^2 = 1$, Gaussian constants cancel. We use:

$$p(y_i \mid x_i, \beta_k) \propto \exp\left(-\frac{1}{2}(y_i - \beta_k x_i)^2\right)$$

Sample 1: $(x_1 = 1, y_1 = 1)$

- Regime 1:

$$(y_1 - \beta_1 x_1)^2 = (1 - 1)^2 = 0 \Rightarrow e^0 = 1$$

- Regime 2:

$$(y_1 - \beta_2 x_1)^2 = (1 + 1)^2 = 4 \Rightarrow e^{-2} \approx 0.135$$

Posterior:

$$\gamma_{1,1} = \frac{0.5 \cdot 1}{0.5(1 + 0.135)} = \frac{1}{1.135} \approx 0.881$$

Sample 2: $(x_2 = 1, y_2 = -1)$

- Regime 1:

$$(-1 - 1)^2 = 4 \Rightarrow e^{-2} \approx 0.135$$

- Regime 2:

$$(-1 + 1)^2 = 0 \Rightarrow e^0 = 1$$

Posterior:

$$\gamma_{2,1} = \frac{0.5 \cdot 0.135}{0.5(0.135 + 1)} = \frac{0.135}{1.135} \approx 0.119$$

Sample 3: $(x_3 = 1, y_3 = 0)$

- Regime 1:

$$(0 - 1)^2 = 1 \Rightarrow e^{-0.5} \approx 0.606$$

- Regime 2:

$$(0 + 1)^2 = 1 \Rightarrow e^{-0.5} \approx 0.606$$

Posterior:

$$\gamma_{3,1} = \frac{0.5 \cdot 0.606}{0.5(0.606 + 0.606)} = 0.5$$

Summary of E-Step

| Sample $i$ | $\gamma_{i,1}$ | $\gamma_{i,2}$ |
|---|---|---|
| 1 | 0.881 | 0.119 |
| 2 | 0.119 | 0.881 |
| 3 | 0.500 | 0.500 |

## 2. M-Step: Derivation

For each regime $k$, minimize the weighted MSE:

$$J(\beta_k) = \sum_{i=1}^{N} \gamma_{ik} (y_i - \beta_k x_i)^2$$

Take derivative:

$$\frac{\partial J}{\partial \beta_k} = -2 \sum_{i=1}^{N} \gamma_{ik} x_i (y_i - \beta_k x_i)$$

Set equal to zero:

$$\sum_{i=1}^{N} \gamma_{ik} x_i y_i = \beta_k \sum_{i=1}^{N} \gamma_{ik} x_i^2$$

Thus, the update is:

$$\beta_k^{\text{new}} = \frac{\sum_{i=1}^{N} \gamma_{ik} x_i y_i}{\sum_{i=1}^{N} \gamma_{ik} x_i^2}$$

## 3. M-Step: Calculation

Since $x_i = 1$ for all samples:

$$\beta_k^{\text{new}} = \frac{\sum_i \gamma_{ik} y_i}{\sum_i \gamma_{ik}}$$

Update for Regime 1
Numerator:

$$0.881(1) + 0.119(-1) + 0.5(0) = 0.762$$

Denominator:

$$0.881 + 0.119 + 0.5 = 1.5$$

Update:

$$\beta_1^{(1)} = \frac{0.762}{1.5} \approx 0.508$$

Update for Regime 2
Using $\gamma_{i,2} = 1 - \gamma_{i,1}$:
Numerator:

$$0.119(1) + 0.881(-1) + 0.5(0) = -0.762$$

Denominator:

$$1.5$$

Update:

$$\beta_2^{(1)} = \frac{-0.762}{1.5} \approx -0.508$$

## 4. Conceptual Question

OLS Estimate (Single Regression)

Standard OLS without intercept:

$$\hat{\beta}_{\text{OLS}} = \frac{\sum_i x_i y_i}{\sum_i x_i^2} = \frac{1 + (-1) + 0}{3} = 0$$

Why OLS Is Misleading

- OLS **averages across regimes**
- Positive and negative regimes cancel out
- Produces $\beta = 0$, suggesting *no relationship*

Why Mixture Model Is Better

- Identifies **two latent regimes**
- Learns separate slopes:

$$\beta_1 \approx +0.51, \qquad \beta_2 \approx -0.51$$

- Captures **regime-dependent behavior**, which is crucial in finance

Final Interpretation

- OLS ignores heterogeneity ⟶ misleading
- Mixture of regressions captures hidden market states
- EM separates bull and bear dynamics even when regimes are unobserved